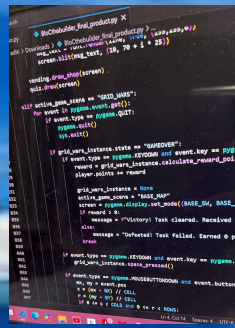


JuneJam

Aditi, Dhillshan, Kaushik, Jaris

BloCthebuilder



Story behind our code:

As Jc students,
life is stressful yet we slack
(Junejam > homework)
So WE decided to code a game to help
US study!!!



This da code for qns:

- Depending on levels $\gg$ difficulty of qns increases
- Dis da options popping up when home menu is clicked



```
1 # — Integrated Task Homework List System
2 class QuizManager:
3     def __init__(self):
4         # Expanded Question pools to support randomization and robust scaling
5         self.subjects = {
6             "Math": {
7                 "Easy": [
8                     ("2 + 2 = ?", "4", "Think basic single-digit addition"),
9                     ("5 - 3 = ?", "2", "Subtract carefully"),
10                    ("10 + 5 = ?", "15", "Simple double-digit addition"),
11                    ("4 * 2 = ?", "8", "Think about adding 4 twice"),
12                    ("9 - 4 = ?", "5", "Count down from 9"),
13                    ("3 + 7 = ?", "10", "Forms a perfect ten")
14                ],
15                "Medium": [
16                    ("12 / 3 = ?", "4", "What multiplied by 3 gives 12?"),
17                    ("sqrt(16) = ?", "4", "What square number gives 16?"),
18                    ("15 * 4 = ?", "60", "Double 15 twice"),
19                    ("7 * 8 = ?", "56", "Classic multiplication table check"),
20                    ("5^2 = ?", "25", "Multiply 5 by itself"),
21                    ("100 - 37 = ?", "63", "Careful with borrowing values")
22                ],
23                "Hard": [
24                    ("Integral of x dx? (Express like 0.5x^2)", "0.5x^2", "Use calculus power rule in"),
25                    ("Derivative of sin(x)?", "cos(x)", "Trigonometric rates of change"),
26                    ("Derivative of x^2? (w.r.t x)", "2x", "Standard power rule application"),
27                    ("Solve for x: 2x + 5 = 15", "5", "Isolate variable by subtracting 5 first"),
28                    ("Log10(100) = ?", "2", "10 raised to what power is 100?"),
29                    ("Derivative of e^x?", "e^x", "The function remains unique and unchanged")
30                ]
31            }
32        }
```

This da use of points (from quizzes)

- Energy drinks:
adds time (5s
and stackable)



- Snacks:
Allow redo of
mistakes (for
bigbacks)



- Hint scroll: give
hint (for
morons)



```
# — Vending Machine —
class VendingMachine:
    def __init__(self):
        self.items = [("Energy Drink", 10), ("Snack", 5), ("Hint Scroll", 15)]
        self.x, self.y, self.w, self.h = 450, 120, 80, 120
        self.color = (255,200,0)
        self.shop_open = False
        self.selected_index = 0

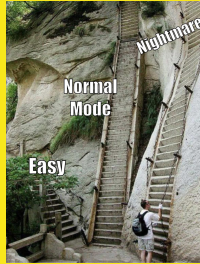
    def draw(self, screen):
        pygame.draw.rect(screen, self.color, (self.x,self.y,self.w,self.h))
        label = font.render("SHOP (E)", True, (0,0,0))
        screen.blit(label, (self.x, self.y - 20))

    def player_nearby(self, player):
        return (player.x < self.x+self.w and player.x+player.size > self.x and
                player.y < self.y+self.h and player.y+player.size > self.y)

    def navigate(self, event, player):
        if self.shop_open and event.type == pygame.KEYDOWN:
            if event.key == pygame.K_UP:
                self.selected_index = (self.selected_index-1) % len(self.items)
            elif event.key == pygame.K_DOWN:
                self.selected_index = (self.selected_index+1) % len(self.items)
            elif event.key == pygame.K_RETURN:
                item,cost = self.items[self.selected_index]
                if player.points >= cost:
                    player.points -= cost
                    player.inventory.append(item)
                    return f"Bought {item}! Remaining points: {player.points}"
```

This da code for minigame (Grid Wars)

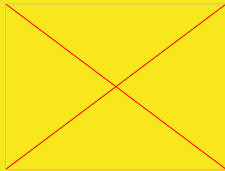
- Modes: Ez 🤔, Norm 🐼, Hard 🐈



- Characters: 7 of em with unique abilities



- About game: Smth like chess but they can shoot



```
# — Integrated Grid Wars Gameplay Context Wrapper  
class IntegratedGridWars:
```

```
    def __init__(self, difficulty_setting):  
        setting_map = {"Easy": "EASY", "Medium": "NORMAL", "Hard": "HARD"}  
        self.diff_key = setting_map.get(difficulty_setting, "NORMAL")  
        self.diff = DIFFICULTY[self.diff_key]  
        self.total_enemies_spawned = self.diff["enemy_count"]  
        self.units = []  
        self._setup()  
        self.state = "PLAYER_SELECT"  
        self.selected = None  
        self.ability_mode = False  
        self.move_tiles, self.shoot_tiles, self.ability_tiles = [], [], []  
        self.bullets, self.bursts, self.floats = [], [], []  
        self.winner = None  
        self.dodge_target = None  
        self.dodge_timer, self.dodge_window = 0.0, 0.0  
        self.dodged = False  
        self.enemy_q = []  
        self.enemy_delay = 0.0  
        self.screen_shake = 0  
  
    def _setup(self):  
        for kind,c,r in [("TANK",1,4),("SOLDIER",1,2),("MEDIC",1,7),("SNIPER",2,0),("GRENADIER",2,9)]:  
            self.units.append(Unit(kind,c,r,"player"))  
        all_e = [("TANK",11,5),("SOLDIER",11,2),("SOLDIER",11,7),("SCOUT",10,0),("SCOUT",10,9),("RECON",10,5)]  
        for kind,c,r in all_e[:self.total_enemies_spawned]:  
            self.units.append(Unit(kind,c,r,"enemy"))
```

THANK YOU



Department of Education